

Projeto 01: Busca Adversarial em Jogos

Agente Inteligente para Othello (Reversi)

Bruno Amaral Salles (281129) • Rodrigo Banin Ferraz de Camargo (238257)
Caio Lima Albulquerque (288808) • João Pedro Bonatti (237575) • Ricardo Andrade Oliveira Bello (247326)

April 1, 2026

1 Introdução - Othello/Reversi

O Othello é um jogo de tabuleiro de estratégia disputado em um Grid 8x8, caracterizado como um problema determinístico, de soma zero e de observabilidade total. A dinâmica baseia-se no flanqueamento e captura das peças do adversário em que a estratégia dominante consiste em garantir o controle sobre os cantos do tabuleiro, uma vez que posições nestes vértices são irreversíveis (ditas "estáveis") e permitem a construção de zonas estáveis adjacentes. Este projeto implementa agentes de busca adversarial que exploram a captura de cantos, restrição de mobilidade do adversário e minimização da exposição de peças de fronteira.

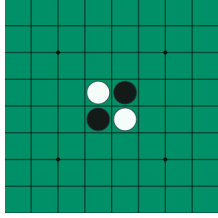


Figure 1: Estado inicial do tabuleiro

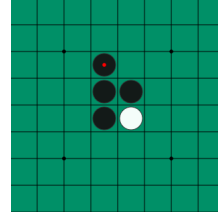


Figure 2: Primeiro movimento das peças pretas

2 Modelagem do Problema

2.1 Definição Formal e Teste de Terminal

O jogo Othello é definido formalmente pela tupla (S, A, T, U) :

- **Estado (S):** Uma matriz 8×8 onde cada posição $B_{i,j} \in \{\emptyset, \text{Preto}, \text{Branco}\}$. O estado inicial S_0 contém 4 peças centrais cruzadas. O turno atual é indicado por $p \in \{\text{Preto}, \text{Branco}\}$.
- **Ações (A):** O conjunto de coordenadas (i, j) onde $B_{i,j} = \emptyset$ e a colocação da peça flanqueia ao menos uma peça oponente em linha reta, terminando em uma peça do jogador p .
- **Transição (T):** Aplica uma ação $a \in A$ ao estado s , invertendo a cor das peças flanqueadas e alternando o turno ($p \leftarrow -p$). Se $A = \emptyset$ para o jogador, o turno é passado sem alterações no tabuleiro.
- **Teste de Terminal e Utilidade (U):** Um estado é terminal quando ambos os jogadores não possuem ações válidas ($A(p) = \emptyset$ e $A(-p) = \emptyset$). A função de utilidade $U(s)$ avalia a vantagem estratégica do jogador p em estados não terminais através de heurísticas (detalhadas na Seção 3).

2.2 Representação de Estado e Geração de Ações

Na implementação computacional, a matriz do tabuleiro utiliza a correspondência numérica: Vazio = 0, Preto = 1 e Branco = -1. O controle de turnos segue a lógica $p \leftarrow p \times (-1)$. A geração de ações $\mathcal{A}(s)$ itera sobre as células de valor 0, validando-as por buscas radiais nas 8 direções. Durante a transição na árvore de busca, a matriz do estado original é clonada para evitar mutações indesejadas, aplicando-se o movimento e invertendo o turno no nó filho gerado.

3 Algoritmos de Busca

3.1 Minimax

O algoritmo Minimax modela o jogo como uma árvore de decisão em que dois jogadores alternam turnos: o jogador MAX busca maximizar a utilidade do estado, enquanto o jogador MIN busca minimizá-la. A premissa é que o adversário joga de forma ótima, de modo que o agente escolhe o lance que garante o melhor resultado no pior cenário. A implementação percorre recursivamente a árvore de jogo até uma profundidade limite d ou até atingir um estado terminal. Nos nós folha, a função heurística $U(s, p)$ é avaliada. Quando um jogador não possui jogadas válidas, o turno é transferido ao oponente. A cada nó visitado, incrementa-se o contador de nós expandidos. A complexidade temporal do Minimax aplicada ao Othello é $O(b^d)$, onde $b \approx 10$ (fator de ramificação médio) e d a profundidade de busca, tornando buscas profundas computacionalmente caras.

3.2 Alpha-Beta Pruning

O algoritmo Alpha-Beta Pruning é uma otimização do Minimax que elimina ramos da árvore que não podem influenciar o resultado. Para isso, mantém dois limites durante a busca:

- α : melhor valor já garantido pelo jogador MAX (limite inferior);
- β : melhor valor já garantido pelo jogador MIN (limite superior).

Sempre que $\beta \leq \alpha$ em um nó, os ramos restantes são podados, pois o jogador racional (MAX ou MIN) jamais permitirá que a busca alcance aquele estado. O impacto da poda pode ser quantificado comparando o número médio de nós expandidos por jogada entre Minimax e Alpha-Beta sob a mesma heurística e orçamento de tempo (0,5s), conforme a Tabela 1.

Heurística	Minimax (nós)	Alpha-Beta (nós)	Redução
Corner	3724,46	1630,73	56,2%
Frontier	1917,55	1327,70	30,8%
Mobility	959,47	850,83	11,3%

Table 1: Redução no número médio de nós expandidos por jogada com Alpha-Beta em relação ao Minimax.

A heurística *corner* apresenta a maior taxa de poda (56,2%) porque a avaliação de cantos produz escores mais discriminativos na ordenação de jogadas, gerando cortes mais precoces. Já a heurística de *mobility* apresenta menor redução (11,3%) pois a contagem de movimentos válidos oscila rapidamente entre estados vizinhos, produzindo uma ordenação menos eficaz.

3.3 Iterative Deepening e Limite de Tempo

Com tempo fixo de 0,5s por jogada, ambos os algoritmos utilizam *Iterative Deepening* combinado com controle temporal. A ideia é executar buscas com profundidade crescente ($d = 1, 2, 3, \dots$), mantendo sempre o melhor lance encontrado na iteração anterior. Caso o tempo se esgote durante uma iteração, ela é abortada e o lance da última iteração completa é retornado. Essa abordagem garante que o agente sempre possui um lance válido para retornar ao esgotar o tempo. A verificação de tempo limite é realizada em cada nó expandido da árvore. No torneio, utilizou-se `tempo_limite = 0,5s` e `profundidade_max = 64`.

3.4 Ordenação de Jogadas (*Move Ordering*)

A eficácia do Alpha-Beta depende da ordem em que os nós filhos são explorados. Quando os melhores lances são avaliados primeiro, as podas ocorrem mais cedo e a busca atinge maior profundidade dentro do mesmo orçamento de tempo. A implementação utiliza uma função de ordenação que, para cada lance legal, simula o movimento resultante e avalia o estado filho com uma heurística rápida (*corner_heuristic*, controle de cantos). Os filhos são então ordenados em ordem decrescente de escore para o jogador MAX e crescente para o jogador MIN.

4 Funções Heurísticas

Foram implementadas três funções de utilidade (U) para avaliar a qualidade dos estados não terminais, permitindo que os algoritmos de busca tomem decisões informadas sob o limite de tempo de 0,5s.

4.1 Controle de Cantos

Considerada a estratégia mais forte no Othello, foca na captura dos quatro vértices do tabuleiro — $C = \{(1, 1), (1, 8), (8, 1), (8, 8)\}$. Discos em cantos são estáveis, pois não podem ser flanqueados. A utilidade bonifica a posse desses espaços e penaliza a posse pelo adversário (I é a função indicadora de posse da célula pelo jogador p ou oponente $\neg p$):

$$U_{corner}(s, p) = \sum_{(i,j) \in C} I(B_{i,j} = p) - \sum_{(i,j) \in C} I(B_{i,j} = \neg p)$$

4.2 Mobilidade

Esta heurística quantifica o grau de liberdade de movimentação, buscando "sufocar" o oponente ao reduzir suas opções de jogo. A utilidade é calculada pela diferença de cardinalidade entre os conjuntos de ações válidas (A) de ambos os jogadores no estado s :

$$U_{mobility}(s, p) = |A(p)| - |A(\neg p)|$$

4.3 Discos de Fronteira

Um disco de fronteira é aquele adjacente a pelo menos uma célula vazia, representando uma peça vulnerável a capturas futuras. A estratégia consiste em minimizar a própria fronteira enquanto se maximiza a do adversário. Seja $\mathcal{F}(s, x)$ o total de discos de fronteira do jogador x :

$$U_{frontier}(s, p) = \mathcal{F}(s, \neg p) - \mathcal{F}(s, p)$$

5 Avaliação Experimental

A fim de avaliar os algoritmos, realizou-se um torneio automatizado em que as combinações algoritmo+heurística enfrentam todas as demais permutações sob as mesmas regras e orçamento computacional. Cada par ordenado de agentes disputou múltiplas partidas, alternando quem joga de pretas e brancas, totalizando 1568 jogos.

Métrica	Valor
Total de jogos	1568
Vitórias (Pretas)	678
Vitórias (Brancas)	848
Empates	42
Tempo máx. por agente (s)	0,5
Profundidade máx. (nós)	64

Table 2: Resumo geral dos jogos

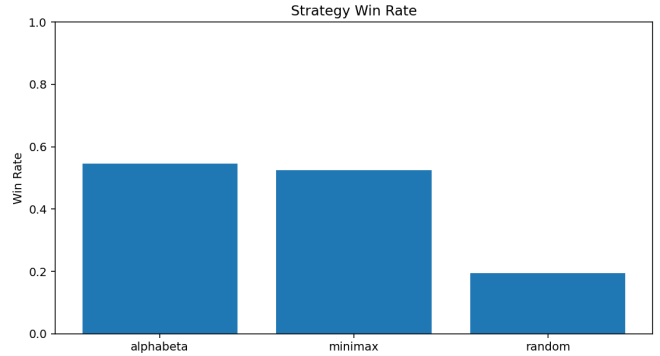


Figure 3: Taxas de vitória por estratégia.

Observa-se na Tabela 2 que as peças brancas venceram mais partidas (848 vs. 678). Esse viés está relacionado à vantagem posicional do segundo jogador no Othello: as brancas reagem ao posicionamento das pretas e frequentemente conseguem melhores posições nos cantos nas fases intermediárias.

5.1 Comparação: Minimax vs Alpha-Beta

A Figura 3 mostra que as taxas de vitória de Minimax e Alpha-Beta são semelhantes. Isso ocorre porque, dentro do orçamento de 0,5s, ambos alcançam profundidades suficientes para tomar decisões competitivas. Entretanto, Alpha-Beta atinge essas decisões de forma substancialmente mais eficiente.

A Tabela 3 detalha a comparação por heurística. Alpha-Beta expande menos nós e alcança maior profundidade média em todos os cenários, com destaque para a heurística de cantos, onde a redução de nós é de 56,2% e o ganho de profundidade é de +1,12 níveis, o que confirma a importância da poda para explorar a árvore de forma mais profunda sem custo adicional de tempo..

5.2 Comparação entre Heurísticas

De acordo com a figura 4 a heurística de **cantos** (corner) é claramente superior, com os agentes *alphabeta+corner* e *minimax+corner* atingindo taxas de vitória de 73,9% e 65,4%, respectivamente. Essa superioridade da heurística de cantos é esperada visto que discos posicionados nos cantos são irrevetíveis, conferindo vantagem estratégica permanente. A partir dos cantos, é possível estabilizar toda uma borda, acumulando peças que não podem ser

Heurística	Nós (Minimax)	Nós (A-B)	Prof. (Minimax)	Prof. (A-B)
Corner	3724,46	1630,73	4,63	5,75
Frontier	1917,55	1327,70	4,31	4,92
Mobility	959,47	850,83	3,78	4,42

Table 3: Nós expandidos e profundidade média por jogada: Minimax vs. Alpha-Beta, por heurística.

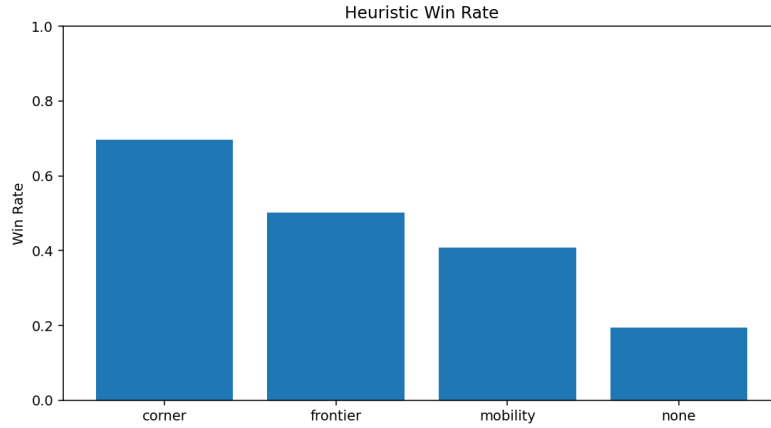


Figure 4: Taxas de vitória por heurística. A heurística de cantos se destaca como a mais eficaz, seguida por mobilidade e fronteira; a ausência de heurística (random) apresenta o pior desempenho.

viradas. A heurística de **fronteira** apresenta desempenho intermediário (54,5% para minimax, 45,8% para alpha-beta). Sua estratégia de minimizar discos expostos é defensivamente razoável, mas não captura oportunidades ofensivas como a posse de cantos. A heurística de **mobilidade** apresenta resultados inferiores (37,7% para minimax, 44,0% para alpha-beta). A contagem de movimentos válidos é uma métrica ruidosa: flutua rapidamente entre estados vizinhos e não reflete diretamente o valor posicional das peças no tabuleiro.

5.3 Comparação entre Agentes

Cada agente é composto por uma estratégia + heurística. Como *baseline*, usamos um agente que seleciona jogadas aleatoriamente.

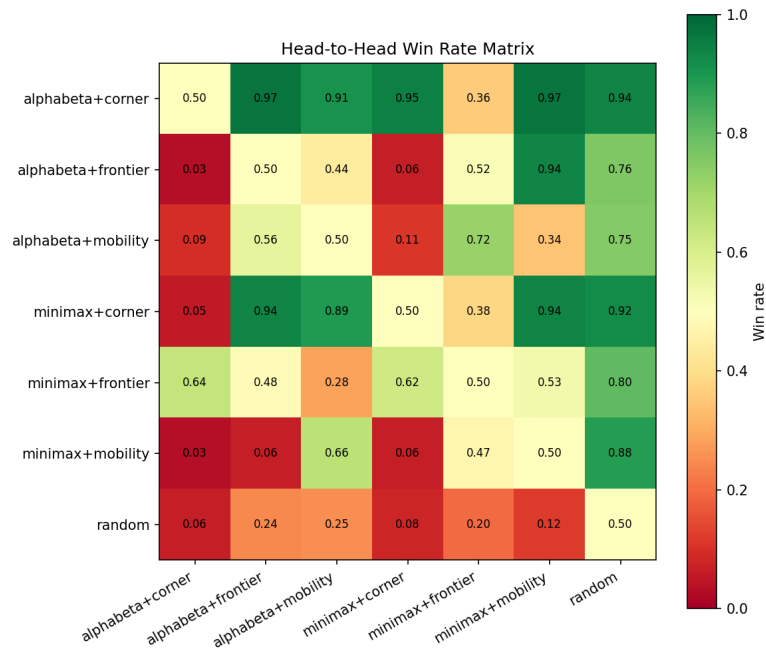


Figure 5: Matriz de taxa de vitória entre todos os pares de agentes. Estratégias informadas superam consistentemente o agente *random* e apresentam desempenho equilibrado entre si, com exceções notáveis.

A Tabela 4 e a Figura 5 revelam que o agente *alphabeta+corner* é o mais forte do torneio, com taxa de vitória de 73,9% e diferença média de discos de +15,65. Todos os agentes com estratégia+heurística superam significativamente

Agente	Wins	Draws	Losses	Win Rate	Disc Diff	Nodes	Depth	Time (s)
alphabet+corner	331	53	64	0,739	15,65	1630,73	5,75	0,09493
alphabet+frontier	205	5	238	0,458	-1,92	1327,70	4,92	0,09553
alphabet+mobility	197	0	251	0,440	-0,37	850,83	4,42	0,09514
minimax+corner	293	6	149	0,654	9,97	3724,46	4,63	0,09542
minimax+frontier	244	6	198	0,545	1,13	1917,55	4,31	0,09590
minimax+mobility	169	3	276	0,377	-7,10	959,47	3,78	0,09566
random	87	11	350	0,194	-17,35	0,00	0,00	0,00000

Table 4: Desempenho detalhado dos agentes

o *baseline* aleatório (19,4%), validando a eficácia da busca adversarial.

A diferença média de discos (*disc diff*) correlaciona-se com a taxa de vitória: os agentes de cantos apresentam os maiores valores positivos, enquanto o agente *random* acumula déficit de -17,35 discos por partida. O tempo médio por jogada é de aproximadamente 0,095s para todos os agentes informados, bem dentro do orçamento de 0,5s, confirmando alocação justa de recursos computacionais.

5.4 Profundidade Média e Nós Expandidos

Analizando a tabela 4 demonstra que o agente *alphabet+corner* atinge a maior profundidade média (5,75) enquanto expande um número moderado de nós (1630,73 por jogada). Em contraste, *minimax+corner* expande o maior número de nós do torneio (3724,46) mas alcança profundidade inferior (4,63), em que a diferença de níveis é atribuível às podas do alpha-beta.

Com fator de ramificação médio $b \approx 10$ no Othello e limite de 0,5s, cada nível adicional requer $\sim 10\times$ mais processamento. As podas do Alpha-Beta efetivamente reduzem o fator de ramificação efetivo, permitindo explorar mais níveis no mesmo intervalo de tempo. Outro dado relevante é que a heurística de cantos, além de produzir melhores avaliações, também gera melhor ordenação de jogadas (pois é usada como heurística rápida de ordenação).

6 Discussão e Conclusões

6.1 Gargalos Computacionais e Limitações Observadas

O principal gargalo temporal concentra-se na cópia da representação matricial de estados e na lógica de validação do motor de jogo. A cada nó expandido, clona-se o tabuleiro inteiro e, em seguida, `valid_moves` percorre todas as células testando 8 direções por célula para determinar as jogadas possíveis. Apesar dessas limitações, os agentes alcançam relativa profundidade na árvore (3,78 a 5,75 níveis) dentro do período de 0,5s, demonstrando que a implementação é funcional para os objetivos do trabalho. A profundidade limitada é parcialmente compensada pela qualidade das heurísticas.

6.2 Relação entre Profundidade de Busca e Qualidade de Jogo

No geral, Alpha-Beta atinge maior profundidade que o Minimax sob o mesmo orçamento temporal, com ganho médio de +0,79 níveis. Entretanto, o agente *minimax+corner*, com profundidade média de 4,63, obtém taxa de vitória de 65,4% — superior ao *alphabet+frontier* (profundidade 4,92, taxa de vitória 45,8%). Isso demonstra que a qualidade da heurística domina o efeito da profundidade adicional, de modo que é mais vantajoso explorar menos níveis com uma boa heurística do que mais níveis com uma heurísticas ruidosas.

Heurística	Prof. Minimax	Prof. Alpha-Beta	Δ Profundidade
Corner	4,63	5,75	+1,12
Frontier	4,31	4,92	+0,61
Mobility	3,78	4,42	+0,64

Table 5: Profundidade média por jogada: Minimax vs. Alpha-Beta.

Os retornos decrescentes observados explicam-se pelo fator de ramificação do Othello ($b \approx 10$): cada nível adicional multiplica o espaço de busca por $\sim 10\times$, de modo que os ganhos da poda Alpha-Beta são rapidamente consumidos. A combinação ideal, confirmada pelos resultados, é um algoritmo de poda eficiente (Alpha-Beta) aliado a uma heurística com forte correlação ao valor real do estado (controle de cantos).

6.3 Conclusão

Este trabalho implementou e avaliou agentes de busca adversarial para o jogo Othello, utilizando os algoritmos Minimax e Alpha-Beta Pruning com *Iterative Deepening*, ordenação de jogadas e três funções heurísticas distintas (mobilidade, controle de cantos e discos de fronteira). O torneio automatizado com 1568 partidas demonstrou que:

1. O agente *alphabet+corner* é o mais forte, com taxa de vitória de 73,9% e diferença média de +15,65 discos por partida.

2. Alpha-Beta reduz o número de nós expandidos em até 56,2% em relação ao Minimax puro, permitindo buscas ~ 1 nível mais profundas no mesmo orçamento de tempo.
3. A heurística de cantos supera as demais independentemente do algoritmo de busca utilizado.
4. A ordenação de jogadas baseada na heurística de cantos potencializa as podas do Alpha-Beta.